BOOK MANAGEMENT SYSTEM

BACKEND IMPLEMENTATION

Server.JS

```
const express = require("express");

const cors = require("cors");

const connectDB = require("./config/db");

const app = express();

connectDB();

app.use(cors());

app.use(express.json());

app.use("/api", require("./routes/auth"));

app.use("/api/books", require("./routes/books"));

app.use("/api/orders", require("./routes/orders"));

app.listen(5000, () => console.log("Server running on port 5000"));
```

Database Connection

```
const mongoose = require("mongoose");

const connectDB = async () => {

  try {

    await mongoose.connect("mongodb://127.0.0.1:27017/bookstore");

    console.log("MongoDB Connected");

  } catch (err) {

    console.error(err);

    process.exit(1);

  }

};
```

```
module.exports = connectDB;
```

User Model

```
const mongoose = require("mongoose");
const userSchema = new mongoose.Schema({
  name: String,
  email: String,
  password: String,
  role: { type: String, default: "user" }
});
module.exports = mongoose.model("User", userSchema);
```

Book Model

```
const mongoose = require("mongoose");
const bookSchema = new mongoose.Schema({
  title: String,
  author: String,
  price: Number,
  stock: Number
});
module.exports = mongoose.model("Book", bookSchema);
```

Order Model

```
const mongoose = require("mongoose");
const orderSchema = new mongoose.Schema({
  userId: String,
  books: Array,
  total: Number,
  status: { type: String, default: "Pending" }
});
module.exports = mongoose.model("Order", orderSchema);
```

Authentication

```
const jwt = require("jsonwebtoken");
const auth = (req, res, next) => {
  const token = req.header("Authorization");
  if (!token) return res.status(401).send("Access denied");
 try {
    const verified = jwt.verify(token, "secret");
    req.user = verified;
    next();
  } catch {
    res.status(400).send("Invalid token");
  }
};
```

```
module.exports = auth;
```

Authentication routes

```
const express = require("express");
const bcrypt = require("bcryptjs");
const jwt = require("jsonwebtoken");
const User = require("../models/User");
const router = express.Router();
router.post("/register", async (req, res) => {
  const hashed = await bcrypt.hash(req.body.password, 10);
  const user = new User({ ...req.body, password: hashed });
  await user.save();
  res.send(user);
});
router.post("/login", async (req, res) => {
  const user = await User.findOne({ email: req.body.email });
  if (!user) return res.send("User not found");
  const valid = await bcrypt.compare(req.body.password, user.password);
  if (!valid) return res.send("Invalid password");
  const token = jwt.sign({ id: user._id, role: user.role }, "secret");
  res.json({ token });
});
module.exports = router;
```

Books Routes

```
const express = require("express");
const Book = require("../models/Book");
const auth = require("../middleware/auth");
const router = express.Router();
router.get("/", async (req, res) => {
  const books = await Book.find();
  res.json(books);
});
router.post("/", auth, async (req, res) => {
  const book = new Book(req.body);
  await book.save();
  res.send(book);
});
module.exports = router;
```

Order Routes

```
const express = require("express");
const Order = require("../models/Order");
const auth = require("../middleware/auth");
const router = express.Router();
router.post("/", auth, async (req, res) => {
  const order = new Order(req.body);
```

```
  await order.save();
  res.send(order);
});
router.get("/", auth, async (req, res) => {
  const orders = await Order.find();
  res.json(orders);
});
module.exports = router;
```

FRONTEND IMPLEMENTSTION

<u>API Configaration</u>

```
import axios from "axios";
const API = axios.create({
  baseURL: "http://localhost:5000/api"
});
API.interceptors.request.use((req) => {
  const token = localStorage.getItem("token");
  if (token) req.headers.Authorization = token;
  return req;
});
export default API;
```

<u>APP Routing</u>

```
import { BrowserRouter, Routes, Route } from "react-router-dom";
```

```
import Home from "./pages/Home";
import Login from "./pages/Login";
import Register from "./pages/Register";
function App() {
  return (
    <BrowserRouter>
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/login" element={<Login />} />
        <Route path="/register" element={<Register />} />
      </Routes>
    </BrowserRouter>
  );
}
export default App;
```

Home Page

```
import { useEffect, useState } from "react";
import API from "../api/api";
function Home() {
  const [books, setBooks] = useState([]);
  useEffect(() => {
    API.get("/books").then(res => setBooks(res.data));
  }, []);
  return (
    <div>
```

```
      <h1>Books</h1>
      {books.map(b => (
        <div key={b._id}>
          <h3>{b.title}</h3>
          <p>{b.author}</p>
          <p>₹{b.price}</p>
        </div>
      ))}
    </div>
  );
}
export default Home;
```

Login Page

```
import { useState } from "react";
import API from "../api/api";
import { useNavigate } from "react-router-dom";
function Login() {
  const [form, setForm] = useState({});
  const navigate = useNavigate();
  const handleLogin = async () => {
    const res = await API.post("/login", form);
    localStorage.setItem("token", res.data.token);
    navigate("/");
  };
  return (
```

```
    <div>
      <input placeholder="Email" onChange={e => setForm({...form, email: e.target.value})} />
      <input type="password" placeholder="Password" onChange={e => setForm({...form,
password: e.target.value})} />
      <button onClick={handleLogin}>Login</button>
    </div>
  );
}
export default Login;
```

Registration Page

```
import { useState } from "react";
import API from "../api/api";
function Register() {
  const [form, setForm] = useState({});
  const handleSubmit = async () => {
    await API.post("/register", form);
    alert("Registered Successfully");
  };
  return (
    <div>
      <input placeholder="Name" onChange={e => setForm({...form, name: e.target.value})} />
      <input placeholder="Email" onChange={e => setForm({...form, email: e.target.value})} />
      <input type="password" placeholder="Password" onChange={e => setForm({...form,
password: e.target.value})} />
      <button onClick={handleSubmit}>Register</button>
    </div>
```

```
  );
}
export default Register;
```